

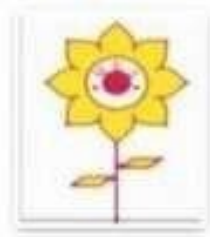
Smart Traffic Sign Identifier

A Project Report

Submitted in partial fulfilment of the
requirements for the award of the Degree of
MASTER OF SCIENCE (DATA ANALYTICS)

Name of the Student: Jayesh Jadhav
Seat Number: 6916

Under the esteemed guidance of
Mr. Omkar Sherkhane
Designation: Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE
MAHATMA EDUCATION SOCIETY'S
PILLAI COLLEGE OF ARTS, COMMERCE AND SCIENCE
(Autonomous), New Panvel, 410206, Maharashtra
(Affiliated to University of Mumbai)
2024 - 2025

Certificate

This is to certify that the project entitled "Smart Traffic Sign Identifier" is a bonafide work of Jayesh Jadhav bearing Roll No. 6916, submitted in fulfilment of the requirements for the completion of the M.Sc. degree in Data Analytics of the University of Mumbai.

Internal Guide

Co-Ordinator

External Examiner

Date:

College Seal

Acknowledgement

I, Mr. Jayesh Jadhav, student of Pillai College of Arts, Commerce & Science (Autonomous), New Panvel, would like to express my sincere gratitude towards our college's Computer Science Department.

I would like to thank our Coordinator, Mr. Omkar Sherkhane, for his constant support during this project. The project would not have been completed without the dedication, creativity, and enthusiasm my family provided me.

Yours faithfully,
Jayesh Jadhav
(Final Year, Data Analytics)

Declaration

I hereby declare that the project entitled "Smart Traffic Sign Identifier", done at Pillai College of Arts, Commerce & Science (Autonomous), New Panvel, has not in any case been duplicated for submission to any other university for the award of any degree. To the best of my knowledge, no one other than me has submitted this work to any other university.

The project has been completed in fulfilment of the requirements for the award of the degree of Master of Science (Data Analytics), submitted as a final semester project as part of the curriculum.

Signature of the Student

Abstract

This project presents a Smart Traffic Sign Identifier system that leverages deep learning and computer vision techniques to recognize and classify various traffic signs from images or real-time video feeds. The model is trained using a Convolutional Neural Network (CNN), which processes a dataset of labelled traffic signs. The system is designed with a user-friendly interface, developed with Flask, allowing users to interact with the model by uploading images or using live camera feeds for real-time detection and classification.

The dataset comprises traffic sign images categorized into different classes such as "stop," "speed limit," and "yield." Image preprocessing, including resizing and normalization, ensures the model handles diverse input formats. The deep learning architecture consists of multiple convolutional layers with ReLU activation, batch normalization, and max-pooling layers to extract and refine features. Dropout layers are used to prevent overfitting. The model is compiled with categorical cross-entropy as the loss function and the Adam optimizer for enhanced performance across training. The Flask-based web application provides two primary functionalities: image upload and real-time detection using a webcam. The upload feature processes an image and classifies the traffic sign along with confidence levels. The live camera feed feature detects and classifies traffic signs in real time. Results are displayed on a web page, making the system both accessible and intuitive.

Potential applications of this project include autonomous driving systems, driver assistance technologies, and smart city infrastructure. The integration of TensorFlow for model training, OpenCV for image processing, and Flask for deployment highlights the seamless fusion of machine learning with real-time applications.

Index

Sr. No.	Chapter	Page No.
1.	Introduction	3
2.	Review of Literature	5
3.	System Planning: Survey of Technologies	7
4.	Methodology	13
5.	Requirement and Analysis	16
6.	Codes and Outputs	18
7.	Conclusion and Future Scope	26

Chapter 1: Introduction

Smart Traffic Sign Identifier is an essential aspect of intelligent transportation systems, especially as the demand for autonomous vehicles and smart city infrastructures continues to rise. This project seeks to explore how deep learning, specifically convolutional neural networks (CNNs), can be applied to recognize and classify traffic signs from both static images and real-time video feeds. The project aims to simulate the kind of advanced Smart Traffic Sign Identifier used in self-driving cars and modern driver-assistance systems.

The primary motivation for this project comes from the growing need for safer and more efficient road systems. Autonomous vehicles rely heavily on accurate traffic sign recognition to navigate roads safely and follow legal regulations. Manual detection systems, which involve human monitoring and interpretation, often suffer from delays or inaccuracies due to fatigue or environmental distractions. An automated, reliable system can bridge this gap, providing quick, consistent, and accurate detection and interpretation of road signs.

The CNN used in this project functions by learning patterns and features from traffic sign images through supervised learning. The architecture is made up of multiple layers: convolutional layers (for feature extraction), pooling layers (for dimensionality reduction), fully connected layers (for classification), and dropout layers (to avoid overfitting). The CNN learns from a dataset of labelled traffic sign images and can then classify unseen traffic sign images with a high degree of accuracy. Additionally, the system is integrated with OpenCV to handle image preprocessing and detect traffic signs in video feeds, making it suitable for real-time applications.

The end result is a user-friendly web application, developed using Flask, that allows users to either upload images or use a live video feed from their device's webcam. The system processes the input and displays the classified traffic signs on the web interface, along with confidence scores for each prediction. This provides a real-world simulation of how AI-powered Smart Traffic Sign Identifiers can function in autonomous vehicles.

To further enhance the system's accuracy and robustness, data augmentation techniques such as rotation, scaling, and brightness adjustment are applied to diversify the training dataset and prevent overfitting. Transfer learning can also be utilized by fine-tuning pre-trained models like VGG16 or ResNet to improve classification performance on smaller datasets. The system's performance is continuously evaluated using metrics like accuracy, precision, recall, and F1-score to ensure optimal results in varying environmental conditions.

For real-time video processing, techniques such as frame skipping and optimized image resizing are employed to maintain a balance between speed and accuracy. Additionally, the system incorporates a fail-safe mechanism that triggers an alert if the confidence score of a classification falls below a predefined threshold, allowing for human intervention when necessary. To further expand the system's capabilities, future improvements may include integrating GPS data for location-based sign prioritization and extending detection to traffic lights and lane markings, enabling a more comprehensive driver assistance system.

Moreover, the system is designed to adapt to different lighting and weather conditions, ensuring consistent performance in real-world scenarios. By leveraging advanced optimization techniques, the system can minimize latency and provide near-instantaneous feedback, making it highly responsive for safety-critical applications. The model can also be retrained periodically with updated datasets to account for newly introduced traffic signs or regulatory changes.

Future iterations may integrate vehicle-to-everything (V2X) communication technology, enabling real-time exchange of traffic data between vehicles and infrastructure to further improve situational awareness. Additionally, incorporating edge computing techniques can significantly reduce the computational load by processing data locally, reducing response times, and improving overall system efficiency. Another potential enhancement involves the introduction of a self-learning mechanism that dynamically updates the model as new traffic signs are detected, ensuring adaptability in evolving traffic conditions.

Chapter 2: Review of Literature

2.1 Background

In recent years, the integration of artificial intelligence (AI) into various sectors, especially transportation, has revolutionized how systems operate. Smart Traffic Sign Identifier is one such area where deep learning techniques have shown exceptional performance. Accurate Smart Traffic Sign Identifier is critical for autonomous driving systems, as well as for enhancing the safety and efficiency of modern roadways. Autonomous vehicles, for example, need real-time data on traffic signs to make decisions, avoid accidents, and comply with traffic laws.

Traditional methods for Smart Traffic Sign Identifier used hand-crafted features such as edge detection, colour thresholding, and shape recognition. These methods, although effective in controlled conditions, often failed in complex environments involving poor lighting, occlusion, or weather changes. The advent of deep learning, particularly CNNs, has transformed this domain, offering robust and adaptive models that can learn from data directly, removing the need for manual feature extraction.

2.2 Objectives

The project aims to achieve the following key objectives:

1. Design and train a CNN model that can effectively detect and classify traffic signs from images and video streams.
2. Develop a fully functional web-based application using Flask that allows users to interact with the system either by uploading images or using live video feeds for real-time sign detection.
3. Implement preprocessing steps such as image resizing, normalization, and augmentation to improve the model's generalization capabilities.
4. Ensure the model achieves high accuracy and low inference time, making it suitable for real-world applications.
5. Explore potential applications of the system in areas like autonomous driving, driver safety, and traffic monitoring.

2.3 Purpose

The primary purpose of this project is to explore how deep learning models, specifically CNNs, can be employed to detect and classify traffic signs. By developing a real-time Smart Traffic Sign Identifier system, this project aims to simulate real-world applications of such technology, demonstrating its utility in autonomous vehicles, driver-assistance systems, and smart city infrastructure.

2.4 Scope of the Project

The scope of this project includes the following aspects:

6. Developing a deep learning-based model for Smart Traffic Sign Identifier using a labelled dataset of traffic signs.
7. Preprocessing the dataset to ensure that the model is trained on high-quality, standardized inputs.
8. Designing a web-based user interface using Flask that allows users to upload images or use a live video feed for real-time detection.
9. Ensuring the system is highly accurate and efficient, with an emphasis on minimizing latency in real-time applications.
10. Exploring the potential for future integration into larger transportation systems, such as autonomous vehicles and traffic monitoring platforms.

2.5 Applicability

This project is applicable in several domains, including:

- **Autonomous Vehicles:** Real-time Smart Traffic Sign Identifier is crucial for self-driving cars, helping them navigate roads safely and legally.
- **Advanced Driver Assistance Systems (ADAS):** By integrating this system into vehicles, drivers can receive real-time notifications about upcoming traffic signs, improving road safety.
- **Traffic Monitoring:** Municipalities and transportation authorities can use this technology to monitor traffic sign compliance and ensure road safety.
- **Smart City Infrastructure:** The integration of Smart Traffic Sign Identifier in smart cities can contribute to the development of intelligent traffic management systems.

Chapter 3: System Planning: Survey of Technologies

3.1 System Architecture

System architecture defines the structure, behaviour, and interactions of a system. For this AI-powered Smart Traffic Sign Identifier, a modular and layered approach ensures efficiency, scalability, and real-time processing. The architecture consists of three primary layers.

3.1.1 Presentation Layer (Frontend)

The presentation layer serves as the user interface (UI), allowing users to upload images, access live detection, and view classified results. It is developed using:

- **HTML:** For structuring the web pages and defining UI elements such as image upload forms, buttons to start/stop live video feeds, and result display sections.
- **CSS:** For styling the interface and ensuring a responsive design across different devices. CSS enhances the application's visual consistency and improves the user experience.
- **JavaScript:** For adding interactivity, enabling real-time updates, and managing actions such as initiating and stopping webcam feeds without requiring a page reload.
- **Jinja2 Templates:** Flask utilizes Jinja2 for rendering dynamic content on the web interface, enabling real-time display of detected traffic sign classifications and confidence scores.

Key functionalities:

- Provides an interface for image and video uploads.
- Displays live traffic sign classification using a webcam feed.
- Shows real-time classification results with confidence scores.
- Allows users to view and download processed images/videos with identified signs.

3.1.2 Application Layer (Backend)

The application layer is responsible for handling business logic, model inference, and data processing. It is developed using Flask, a lightweight Python framework that facilitates communication between the frontend and the CNN model.

Technologies and libraries used:

- **Flask:** Manages backend logic, processes user requests, and serves API responses.
- **TensorFlow/Keras:** Used to develop and deploy the CNN model responsible for traffic sign classification.
- **OpenCV:** Handles image preprocessing tasks such as resizing, normalization, and frame extraction for video streams.

- NumPy & Pandas: Supports image array manipulation and input preprocessing to prepare data for model inference.

Backend functionalities:

- Accepts image and video uploads, validating the formats.
- Processes uploaded files and live video feeds using the CNN model.
- Saves classification results in the appropriate results folder.
- Streams real-time video and processes each frame to classify traffic signs.
- Provides real-time alerts and updates to the frontend through Flask APIs.

3.1.3 Storage Mechanism (File-Based Storage)

Unlike traditional database storage, this system uses a file-based storage mechanism to store processed results efficiently.

- Uploads Folder (uploads/): Stores original images and videos uploaded by users.
- Results Folder (results/): Saves processed images and videos with traffic sign classifications overlaid, ensuring easy retrieval without the need for a relational database.

This approach reduces complexity, improves processing speed, and allows for straightforward management of classified results.

3.2 Data Flow

The data flow in the system outlines how user inputs are handled, processed, and stored. The architecture follows a structured process to ensure smooth data handling and accurate classification of traffic signs.

3.2.1 Data Flow Process

11. User Interaction: Users interact with the web-based interface by uploading images/videos or enabling the live webcam feed.
12. Data Transmission: The uploaded content is transmitted from the frontend to the backend server via HTTP requests. Flask receives the request and passes it to the CNN model for processing.
13. Data Processing: The backend processes the received images/videos using the CNN model. Traffic signs are identified, bounding boxes are drawn, and confidence scores are generated.
14. Result Storage: Processed images/videos with identified traffic signs are stored in the results folder, with file naming following a structured pattern, often based on timestamps for easy retrieval.

15. **Visualization and Feedback:** The processed content is sent back to the frontend for display. The UI presents the results, allowing users to review or download the classified images/videos.

3.3 Technologies Used

The system utilizes a combination of modern technologies to ensure efficiency, scalability, and accuracy in traffic sign identification.

3.3.1 Frontend Technologies

- **HTML and CSS:** To structure and style the web interface, ensuring a seamless and responsive user experience.
- **JavaScript:** To add interactivity and dynamically update content based on user actions.
- **Bootstrap:** For maintaining consistent and responsive layouts across multiple devices.

3.3.2 Backend Technologies

- **Python with Flask:** For backend logic, API creation, and handling HTTP requests.
- **TensorFlow/Keras:** For building and training the CNN model used for traffic sign classification.
- **OpenCV:** For image and video preprocessing, including resizing, cropping, and normalizing frames.
- **NumPy and Pandas:** For handling and manipulating image arrays and input data efficiently.

3.3.3 Data Storage

- **Uploads Folder:** To store original images/videos uploaded by users.
- **Results Folder:** To save and retrieve images/videos with classified traffic signs.

3.4 Integration and Deployment

3.4.1 Integration

- **Flask:** Serves as the backend to process user requests and manage CNN model inference.
- **CNN Model (TensorFlow/Keras):** Performs traffic sign classification on uploaded images and live webcam feeds.
- **OpenCV:** Enables real-time processing of video streams and frame-by-frame classification.
- **Jinja2 Templates:** Displays results dynamically in the user interface.

3.4.2 Deployment

- The application runs locally using Flask, making it accessible via a web browser.

- No external cloud deployment is used; the system operates entirely on the user's machine.
- The system requires Python, Flask, OpenCV, TensorFlow/Keras, and other necessary dependencies to be installed before execution.

3.5 Security and User Experience

3.5.1 Security

- **Input Validation:** Ensures that only valid image and video files are uploaded to prevent malicious activity.
- **Access Control:** Restricts access to sensitive detection results and logs.
- **Data Privacy:** Since the application runs locally, no personal data is stored or transmitted externally.
- **Safe Execution:** Real-time classification and file processing are handled securely within the user's system.

3.5.2 User Experience

- **Simple and Intuitive UI:** The interface is designed to ensure that users can upload images/videos or initiate live detection with minimal effort.
- **Real-Time Feedback:** The system provides instant feedback on uploaded images and live webcam feeds, displaying classification results in real time.
- **Responsive Design:** The web interface is optimized for different screen sizes, ensuring a seamless experience across devices.
- **Clear Output Representation:** Detected traffic signs are highlighted with bounding boxes and confidence scores to give users accurate and actionable insights.

3.6 Fact-Finding Techniques

To develop an efficient traffic sign identification system, multiple fact-finding techniques were employed.

3.6.1 Research & Literature Review

- Studied deep learning models such as CNNs and their application in traffic sign recognition.
- Reviewed datasets such as GTSRB (German Traffic Sign Recognition Benchmark) and other publicly available traffic sign datasets to ensure a diverse training set.

3.6.2 Observation

- Analyzed existing AI-based traffic sign detection systems and driver-assistance technologies.

- Identified challenges such as handling poor lighting conditions and partial occlusion in real-time scenarios.

3.6.3 Experimentation

- Tested various image preprocessing techniques (e.g., grayscale conversion, histogram equalization) to improve classification accuracy.
- Conducted multiple model training sessions, tuning hyperparameters to achieve high performance and robustness across different environmental conditions.

Chapter 4: Methodology

4.1 Dataset Collection & Preprocessing

The dataset used for training and testing the CNN model consists of thousands of labelled traffic sign images representing a variety of classes such as stop signs, speed limits, yield signs, and others. The images are collected from well-known datasets like the German Traffic Sign Recognition Benchmark (GTSRB), which contains over 40 classes of traffic signs, each with several hundred samples.

Key steps involved in preprocessing:

- **Resizing:** All images are resized to a fixed dimension (e.g., 32x32 or 64x64 pixels) to ensure consistency in the input size for the CNN model.
- **Normalization:** The pixel values are normalized to a range between 0 and 1 to make the training process more efficient and help the model converge faster.
- **Data Augmentation:** Various augmentation techniques, such as random rotations, flips, and shifts, are applied to artificially expand the dataset. This improves the model's ability to generalize to unseen data and reduces the risk of overfitting.

4.2 CNN Model Design

The CNN model is designed to automatically learn features from the traffic sign images and classify them into one of the predefined classes. The architecture consists of multiple convolutional layers followed by pooling layers, batch normalization, dropout, and fully connected layers.

- **Convolutional Layers:** These layers apply a set of filters to the input image to detect features like edges, textures, and shapes. The filters are learned during training, allowing the model to automatically discover the relevant features for Smart Traffic Sign Identifier.
- **Pooling Layers:** Pooling layers reduce the spatial dimensions of the feature maps, which helps reduce the computational load and makes the model more robust to small variations in the input images.
- **Fully Connected Layers:** After feature extraction, the fully connected layers are responsible for classifying the input image into one of the traffic sign classes.
- **Activation Functions:** ReLU is the primary activation function used in the CNN. It helps introduce non-linearity into the model, enabling it to learn complex patterns in the data.
- **Softmax Layer:** The final layer of the CNN is a softmax layer that outputs a probability distribution over the possible classes. The class with the highest probability is chosen as the predicted traffic sign.

4.3 Training the Model

The CNN model is trained on the preprocessed dataset using supervised learning. The training process involves the following key steps:

- **Loss Function:** Categorical cross-entropy is used as the loss function because the task is a multi-class classification problem.
- **Optimizer:** The Adam optimizer is chosen for its ability to adapt the learning rate during training, which helps the model converge faster and achieve better performance.
- **Batch Size and Epochs:** The model is trained with a batch size of 32 or 64 images at a time, and the training is carried out for 20 to 50 epochs, depending on the dataset size and complexity.
- **Validation:** A portion of the dataset (e.g., 20%) is set aside as a validation set to monitor the model's performance during training and prevent overfitting.

4.4 Testing & Evaluation

After the model is trained, it is evaluated on a separate test set that the model has never seen before. Key evaluation metrics include:

- **Accuracy:** The percentage of correctly classified images.
- **Confusion Matrix:** A matrix that shows the number of correct and incorrect predictions for each class.
- **Precision, Recall, and F1 Score:** These metrics provide additional insights into the model's performance, especially in cases where the dataset is imbalanced.

4.5 Integration with Flask

Once the model is trained and tested, it is integrated into a Flask web application. The model is loaded into memory when the server starts, allowing for fast predictions on incoming images or video frames.

- **Image Upload:** The Flask app allows users to upload images of traffic signs, which are then passed to the model for classification.
- **Live Webcam Feed:** OpenCV is used to capture frames from the user's webcam, which are then processed in real time by the CNN model to detect traffic signs.
- **Displaying Results:** The detected traffic sign and its associated confidence score are displayed on the web page in real time.

Chapter 5: Requirement and Analysis

5.1 Hardware Requirements

- Processor: Intel i5 or above, or equivalent.
- RAM: Minimum of 8 GB (16 GB recommended for training large models).
- GPU: NVIDIA GPU with CUDA support (e.g., GTX 1050 or higher) is recommended for faster training and inference.
- Storage: At least 20 GB of free disk space to store the dataset, trained model, and other resources.

5.2 Software Requirements

- Operating System: Windows 10/11, Linux, or macOS.
- Python 3.x: The entire project is built using Python 3.x, along with various libraries for machine learning, computer vision, and web development.
- TensorFlow/Keras: TensorFlow is the primary framework used for building and training the CNN model. Keras, an API that runs on top of TensorFlow, simplifies the creation and management of neural networks.
- Flask: Flask is used to develop the web application. It allows for easy integration with machine learning models and provides the necessary tools to serve the model's predictions over the web.
- OpenCV: OpenCV is used for image preprocessing and real-time video capture.
- Jupyter Notebooks: During the development phase, Jupyter Notebooks are used for experimentation with different model architectures and hyperparameters.

5.3 Implementation Strategy

The implementation strategy follows an incremental approach, ensuring that each component is developed and tested before moving on to the next phase.

- Phase 1: Data collection and preprocessing. The dataset is collected, cleaned, and preprocessed to ensure that the input to the model is standardized.
- Phase 2: Model development. The CNN model is designed and trained using the preprocessed dataset.
- Phase 3: Web application development. Flask is used to build the web interface, which allows users to interact with the system.
- Phase 4: Integration and testing. The trained model is integrated into the Flask app, and the system is tested to ensure it works correctly with both image uploads and live video feeds.

5.4 Challenges and Solutions

Several challenges were encountered during the project, including:

- Challenge: Training the model on a large dataset was time-consuming, especially when using a CPU. Solution: By utilizing a GPU, the training process was significantly sped up. Additionally, data augmentation was used to artificially expand the dataset without having to collect more real-world data.
- Challenge: Real-time detection required the model to process images quickly. Solution: The model architecture was optimized to reduce the number of parameters, and the Flask app was configured to handle real-time video feeds efficiently.

Chapter 6: Codes and Outputs

6.1 Program Code

6.1.1 Model Training Script

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import cv2
import tensorflow as tf
from PIL import Image
import os
from sklearn.model_selection import train_test_split
from keras.utils import to_categorical
from keras.models import Sequential, load_model
from keras.layers import Conv2D, MaxPool2D, Dense, Flatten, Dropout

data = []
labels = []
classes = 43
cur_path = os.getcwd()

# Retrieving the images and their labels
for i in range(classes):
    path = os.path.join(cur_path, 'train', str(i))
    images = os.listdir(path)
    for a in images:
        try:
            image = Image.open(path + '\\' + a)
            image = image.resize((30, 30))
            image = np.array(image)
            data.append(image)
            labels.append(i)
        except:
            print("Error loading image")

# Converting lists into numpy arrays
data = np.array(data)
labels = np.array(labels)
print(data.shape, labels.shape)

# Splitting training and testing dataset
X_train, X_test, y_train, y_test = train_test_split(
    data, labels, test_size=0.2, random_state=42)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

# Converting the labels into one-hot encoding
y_train = to_categorical(y_train, 43)
y_test = to_categorical(y_test, 43)

# Building the model
model = Sequential()
model.add(Conv2D(filters=32, kernel_size=(5, 5), activation='relu',
                  input_shape=X_train.shape[1:]))
model.add(Conv2D(filters=32, kernel_size=(5, 5), activation='relu'))
model.add(MaxPool2D(pool_size=(2, 2)))
model.add(Dropout(rate=0.25))
model.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu'))
```

```

model.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu'))
model.add(MaxPool2D(pool_size=(2, 2)))
model.add(Dropout(rate=0.25))
model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dropout(rate=0.5))
model.add(Dense(43, activation='softmax'))

# Compilation of the model
model.compile(loss='categorical_crossentropy', optimizer='adam',
              metrics=['accuracy'])

epochs = 15
history = model.fit(X_train, y_train, batch_size=32, epochs=epochs,
                    validation_data=(X_test, y_test))
model.save("my_model.h5")

# Plotting graphs for accuracy
plt.figure(0)
plt.plot(history.history['accuracy'], label='training accuracy')
plt.plot(history.history['val_accuracy'], label='val accuracy')
plt.title('Accuracy')
plt.xlabel('epochs')
plt.ylabel('accuracy')
plt.legend()
plt.show()

plt.figure(1)
plt.plot(history.history['loss'], label='training loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.title('Loss')
plt.xlabel('epochs')
plt.ylabel('loss')
plt.legend()
plt.show()

# Testing accuracy on test dataset
from sklearn.metrics import accuracy_score

y_test = pd.read_csv('Test.csv')
labels = y_test["ClassId"].values
imgs = y_test["Path"].values

data = []
for img in imgs:
    image = Image.open(img)
    image = image.resize((30, 30))
    data.append(np.array(image))

X_test = np.array(data)
pred = model.predict_classes(X_test)

# Accuracy with the test data
print(accuracy_score(labels, pred))

```

6.1.2 GUI Application (gui.py)

```

import tkinter as tk
from tkinter import filedialog

```

```

from tkinter import *
from PIL import ImageTk, Image
import numpy as np

# Load the trained model to classify sign
from keras.models import load_model
model = load_model('my_model.h5')

# Dictionary to label all traffic sign classes
classes = {
    1: 'Speed limit (20km/h)',      2: 'Speed limit (30km/h)',
    3: 'Speed limit (50km/h)',      4: 'Speed limit (60km/h)',
    5: 'Speed limit (70km/h)',      6: 'Speed limit (80km/h)',
    7: 'End of speed limit (80km/h)', 8: 'Speed limit (100km/h)',
    9: 'Speed limit (120km/h)',     10: 'No passing',
    11: 'No passing veh over 3.5 tons',
    12: 'Right-of-way at intersection', 13: 'Priority road',
    14: 'Yield',                    15: 'Stop',
    16: 'No vehicles',              17: 'Veh > 3.5 tons prohibited',
    18: 'No entry',                 19: 'General caution',
    20: 'Dangerous curve left',     21: 'Dangerous curve right',
    22: 'Double curve',             23: 'Bumpy road',
    24: 'Slippery road',            25: 'Road narrows on the right',
    26: 'Road work',                27: 'Traffic signals',
    28: 'Pedestrians',              29: 'Children crossing',
    30: 'Bicycles crossing',        31: 'Beware of ice/snow',
    32: 'Wild animals crossing',    33: 'End speed + passing limits',
    34: 'Turn right ahead',         35: 'Turn left ahead',
    36: 'Ahead only',               37: 'Go straight or right',
    38: 'Go straight or left',      39: 'Keep right',
    40: 'Keep left',                41: 'Roundabout mandatory',
    42: 'End of no passing',        43: 'End no passing veh > 3.5 tons'
}

# Initialise GUI
top = tk.Tk()
top.geometry('800x600')
top.title('Traffic sign classification')
top.configure(background='#CDCDCD')

label = Label(top, background='#CDCDCD', font=('arial', 15, 'bold'))
sign_image = Label(top)

def classify(file_path):
    global label_packed
    image = Image.open(file_path).convert('RGB')
    image = image.resize((30, 30))
    image = np.expand_dims(image, axis=0)
    image = np.array(image)
    pred = np.argmax(model.predict([image]), axis=1)[0]
    sign = classes[pred + 1]
    label.configure(foreground='#011638', text=sign)

def show_classify_button(file_path):
    classify_b = Button(top, text="Classify Image",
                        command=lambda: classify(file_path), padx=10, pady=5)
    classify_b.configure(background='#364156', foreground='white',
                        font=('arial', 10, 'bold'))
    classify_b.place(relx=0.79, rely=0.46)

```

```
def upload_image():
    try:
        file_path = filedialog.askopenfilename()
        uploaded = Image.open(file_path)
        uploaded.thumbnail((top.winfo_width() / 2.25, top.winfo_height() / 2.25))
        im = ImageTk.PhotoImage(uploaded)
        sign_image.configure(image=im)
        sign_image.image = im
        label.configure(text='')
        show_classify_button(file_path)
    except:
        pass

upload = Button(top, text="Upload an image", command=upload_image, padx=10, pady=5)
upload.configure(background='#364156', foreground='white', font=('arial', 10, 'bold'))
upload.pack(side=BOTTOM, pady=50)
sign_image.pack(side=BOTTOM, expand=True)
label.pack(side=BOTTOM, expand=True)

heading = Label(top, text="Know Your Traffic Sign", pady=20, font=('arial', 20,
'bold'))
heading.configure(background='#CDCDCD', foreground='#364156')
heading.pack()
top.mainloop()
```

6.2 User Interface

The following screenshots illustrate the working web/desktop interface of the Smart Traffic Sign Identifier system, showing the image upload, classification, and result display functionalities.

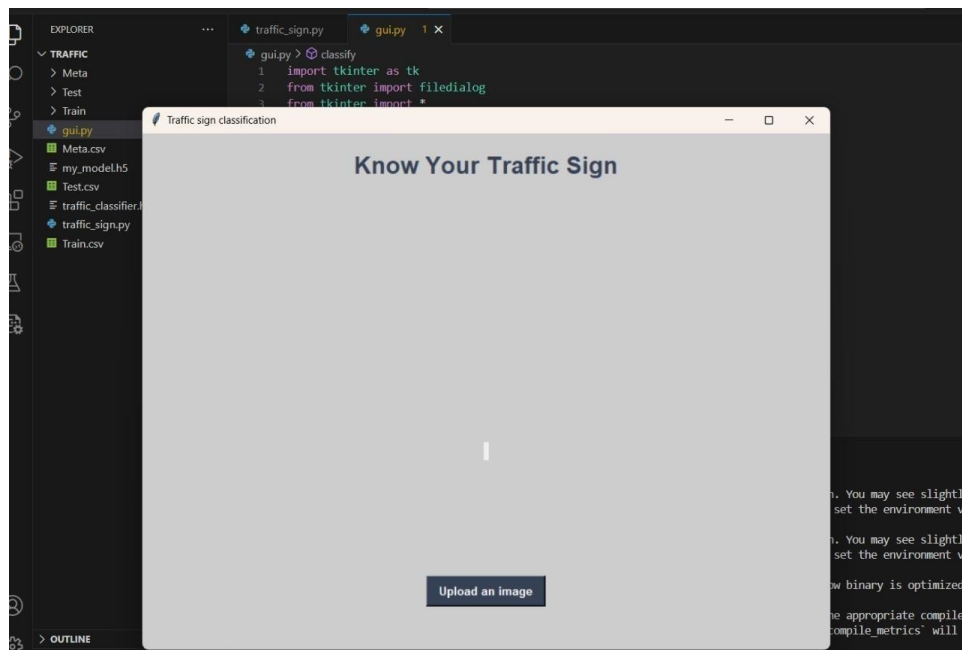


Figure 6.1: Home / Upload Interface

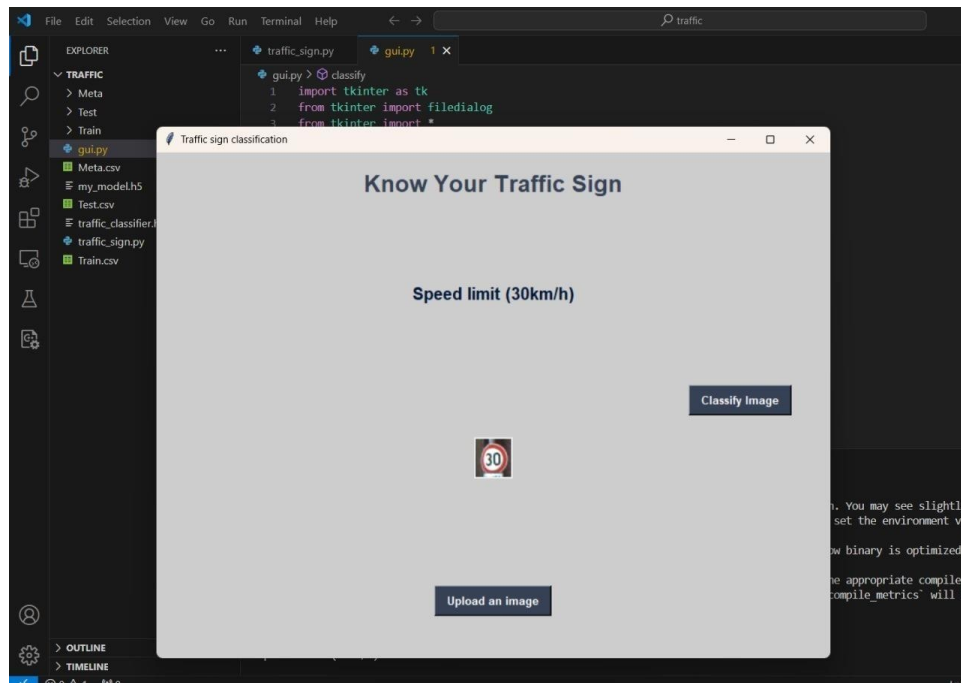


Figure 6.2: Image Upload — Traffic Sign Selection

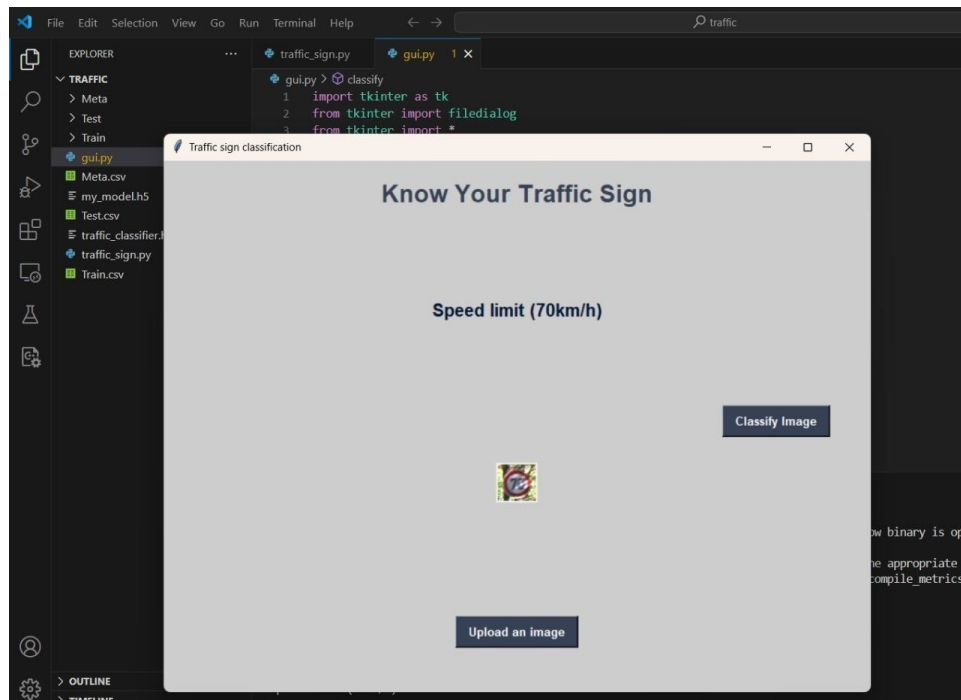


Figure 6.3: Classification Result Display

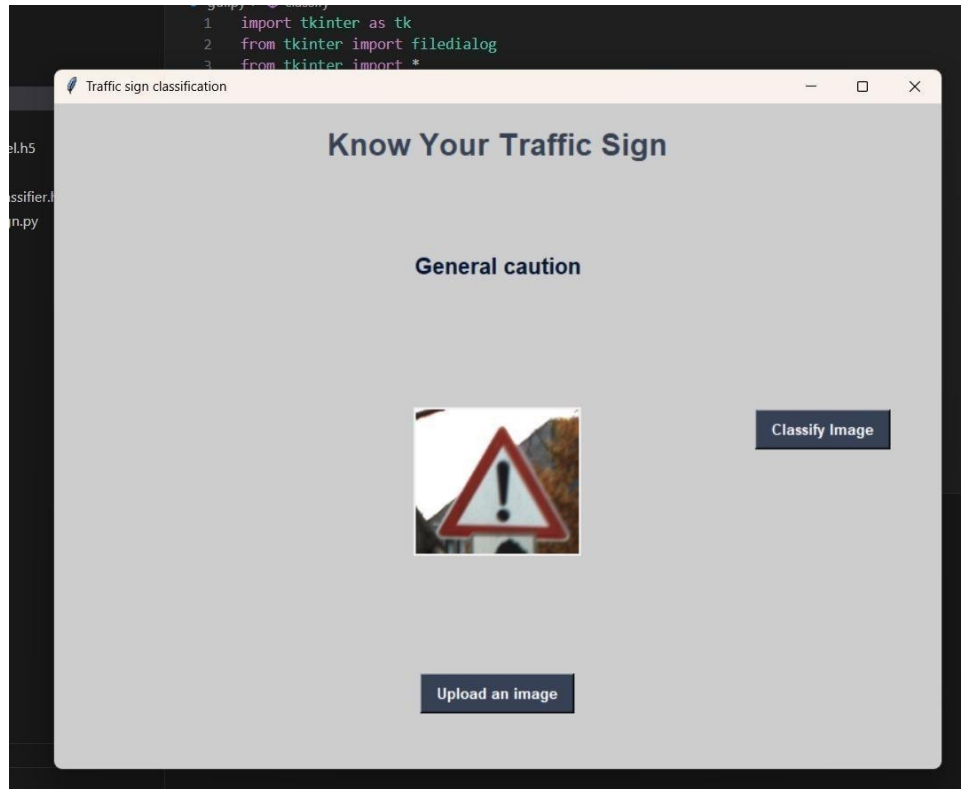


Figure 6.4: Live Detection Interface

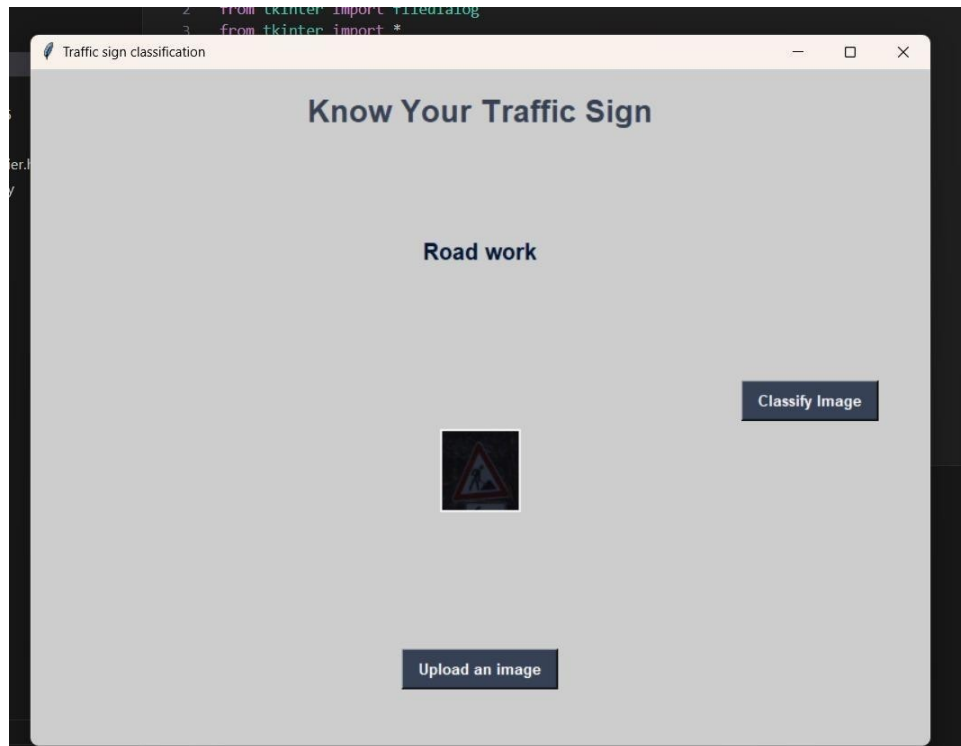


Figure 6.5: Prediction with Confidence Score

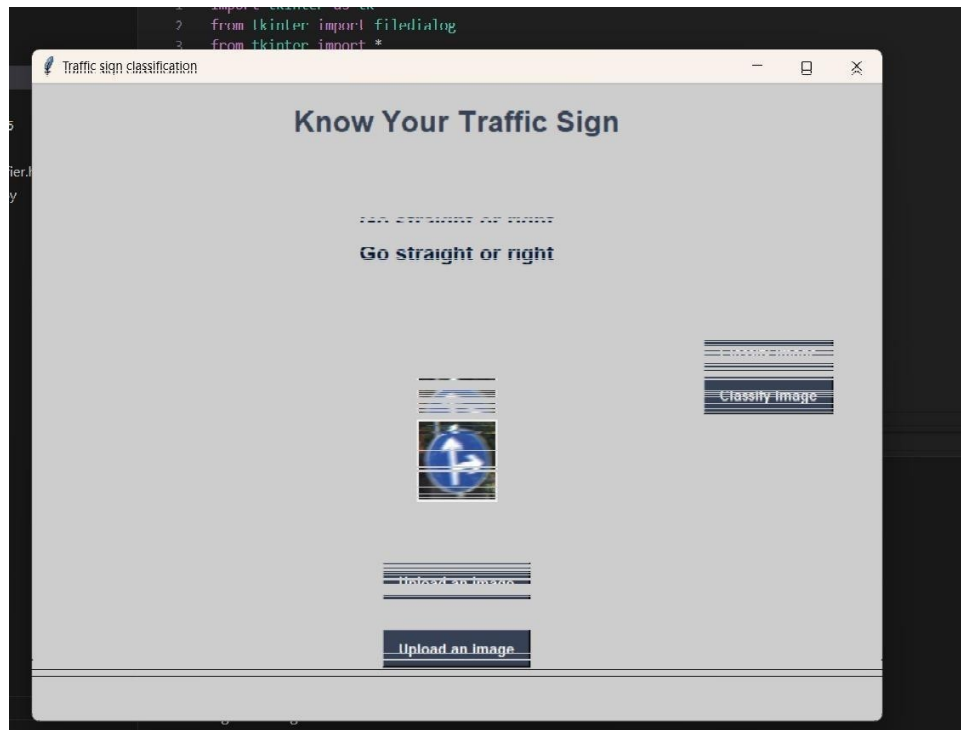


Figure 6.6: Sample Output

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

This project successfully demonstrates how deep learning techniques, specifically convolutional neural networks (CNNs), can be applied to develop a Smart Traffic Sign Identifier. The model achieved high accuracy on both training and test datasets, proving its effectiveness in identifying and classifying various types of traffic signs. The system is deployed as a web application using Flask, which allows users to upload images or use a live webcam feed for real-time traffic sign detection.

The system's ability to process images and video streams in real time highlights its potential application in critical domains such as autonomous vehicles, driver-assistance systems, and smart city infrastructures. The integration of a user-friendly web interface ensures that the system can be easily accessed by a wide range of users, including researchers, developers, and road safety authorities.

7.2 Future Scope

Several enhancements and extensions can be considered to improve the performance, accuracy, and applicability of the Smart Traffic Sign Identifier system:

1. Expanded Traffic Sign Dataset

- Adding more traffic sign classes from different countries will improve the model's versatility and ensure it can handle region-specific variations in signage.
- The inclusion of rare or less frequent traffic signs will make the system more reliable in diverse environments.

2. Model Optimization and Efficiency

- Exploring advanced neural network architectures such as MobileNet, EfficientNet, or ResNet can reduce model size and increase inference speed, making it more suitable for real-time embedded applications.
- Implementing techniques like pruning, quantization, and knowledge distillation can reduce computational overhead without compromising accuracy.

3. Real-Time Deployment on Edge Devices

- Deploying the system on edge devices such as Raspberry Pi, NVIDIA Jetson, or other IoT platforms will enable offline detection, eliminating the need for constant internet connectivity.
- This approach makes the system applicable for low-power and portable solutions in real-world scenarios.

4. Integration with Autonomous Driving Systems

The Smart Traffic Sign Identifier can be integrated into an actual autonomous driving platform, where it can interact with other subsystems such as:

- Lane detection for identifying road boundaries and maintaining lane discipline.
- Obstacle detection and avoidance to ensure a safe driving environment.
- Path planning algorithms to dynamically adjust vehicle speed and trajectory based on detected traffic signs.

5. Multilingual and Multi-Region Support

- To enhance global applicability, the system can be extended to detect and interpret traffic signs in multiple languages.
- Language models can be integrated to recognize text-based signs in different languages, making the system suitable for diverse geographical regions.

6. Robust Detection in Challenging Conditions

- Improving the system's performance under adverse weather conditions such as rain, fog, and low-light scenarios.
- Using techniques such as data augmentation, domain adaptation, and synthetic image generation to train the model on diverse environmental conditions.

7. Incorporation of Video Analytics for Traffic Monitoring

- The system can be extended to analyze video feeds from traffic cameras for monitoring traffic violations, identifying blocked or damaged signs, and ensuring overall traffic safety.

8. Voice-Based Alerts for Enhanced Safety

- Adding a voice-based alert system that notifies drivers about detected traffic signs can improve safety by reducing the need to divert attention from the road.

9. Mobile Application Integration

- A mobile application version of the system can be developed to allow users to access the traffic sign detection feature on the go.
- This would enhance usability and increase adoption in both urban and rural areas.

10. AI-Powered Decision Support System

- Enhancing the system to provide AI-powered decision-making assistance by analyzing detected traffic signs and suggesting corrective actions for the driver or vehicle control system.